

GUIA TÉCNICO · EDIÇÃO 2026

Spec-Driven Development

Como estruturar o desenvolvimento com IA para projetos que escalam — sem cair em teatro de processo.

Departamento WEB · ETAPA Educacional

Versão revisada e expandida

EARS · GitHub Spec Kit · AWS Kiro · Cursor · Claude Code · Copilot

Baseado em padrão EARS (Mavin, Rolls-Royce, 2009),
GitHub Spec Kit (set/2025) e AWS Kiro (jul/2025).

Guia Técnico — Spec-Driven Development (SDD)

Versão revisada e expandida · 2026

Como estruturar o desenvolvimento com IA para projetos que escalam, sem cair em teatro de processo.

Sumário

1. O que é Spec-Driven Development
 2. O problema real que o SDD resolve (e o que ele não resolve)
 3. SDD vs Vibe Coding vs TDD
 4. Quando usar — e quando não usar
 5. EARS — os cinco padrões completos
 6. Anti-padrões em EARS
 7. Estrutura de pastas — versão simplificada e versão Spec Kit oficial
 8. Configuração por ferramenta
 9. Workflow completo — passo a passo
 10. Exemplo end-to-end — API de autenticação
 11. Decomposição em tasks atômicas — critérios mensuráveis
 12. Verificação contra a spec — testes derivados de EARS
 13. Governança — quem mantém o quê
 14. Métricas e sinais de saúde do processo
 15. Referência rápida
-

1. O que é Spec-Driven Development

Spec-Driven Development (SDD) é uma metodologia em que uma **especificação formal é o artefato durável de valor**, e o código passa a ser a saída de build dessa especificação — analogamente a como `.c` compila para binário.

A spec atua como **fonte única de verdade** para três consumidores simultâneos:

- **O agente de IA**, que lê a spec para gerar/modificar código sem precisar reinferir contexto a cada sessão.
- **Os testes**, derivados diretamente de cada cláusula da spec (cada `WHEN/SHALL` vira no mínimo um caso de teste).
- **A revisão humana**, que deixa de perguntar "esse código está bem escrito?" e passa a perguntar "esse código implementa a spec?".

A diferença em relação a uma documentação tradicional é estrutural, não cosmética: a spec SDD é escrita em **linguagem natural restrita** (EARS), com gatilho, condição, ação e payload explícitos. Zero margem para interpretação divergente entre dev, IA e QA.

Origem do termo

O modelo moderno consolidou-se em 2025 com três marcos:

- **AWS Kiro** (julho/2025) — IDE agêntico baseado em Code OSS, adotou EARS como notação nativa de requisitos.
- **GitHub Spec Kit** (setembro/2025) — toolkit open-source, model-agnostic, com CLI `specify` para scaffolding.
- **BMAD-METHOD** — framework comunitário de prompts estruturados.

EARS é mais antigo: criado por **Alistair Mavin na Rolls-Royce em 2009** para requisitos de sistemas críticos (aviação, automotivo, dispositivos médicos).

2. O problema real que o SDD resolve (e o que ele não resolve)

O que resolve

- **Débito de contexto entre sessões de IA.** Agentes não têm memória persistente — sem spec, você re-explica arquitetura toda sessão.
- **Drift de interpretação entre humanos.** Front, back e QA passam a ler o mesmo contrato.
- **Escopo descontrolado em features complexas.** A task atômica deriva da spec, não da vontade do agente.
- **Revisão de código sem critério objetivo.** A pergunta vira binária: implementa a spec ou não.

O que não resolve

***Atenção crítica:** SDD não conserta requisitos ruins. Ele **relocaliza** a ambiguidade. Spec vaga gera código vago do mesmo jeito — só que agora a vagueza está em markdown, com aparência de rigor.*

O ganho real do SDD depende de duas disciplinas que ele **não impõe sozinho**:

1. **Rigor na escrita EARS** — sem os cinco padrões aplicados corretamente, a "spec" vira user story disfarçada.
2. **Sincronização spec ↔ código** — se não há regra dura de "spec e código mudam no mesmo PR", o processo degrada em semanas.

Se a equipe não tem essas duas disciplinas, vibe coding consciente é melhor que SDD performativo.

3. SDD vs Vibe Coding vs TDD

Critério	Vibe Coding	SDD	TDD
Ponto de partida	Intenção em linguagem natural	Spec EARS sem ambiguidade	Teste que falha
Artefato durável	Código	Spec	Suíte de testes
Memória entre sessões	Re-explicar contexto	Agente lê a spec	Testes carregam o contrato
Escopo de erros	Grande e espalhado	Pequeno e localizado na task	Pequeno, mas só no que tem teste
Pergunta da revisão	"Está bem escrito?"	"Implementa a spec?"	"Os testes cobrem o caso?"
Custo inicial	Baixo	Médio (escrever spec)	Médio-alto (escrever testes antes)
Custo em projetos longos	Alto (débito cresce)	Baixo (contexto na spec)	Baixo (regressão sob controle)
Cobertura de regressão	Inexistente	Derivada dos testes da spec	Nativa
Lida bem com IA generativa	Mal	Muito bem	Bem (mas precisa do passo "implementar")
Adequação	Protótipos, scripts	Projetos > 1 sprint	Sistemas críticos

SDD e TDD **não são excludentes** — combinados, cada **SHALL** da spec gera teste, e o teste guia a implementação. Esse é o stack mais robusto para sistemas com SLAs ou compliance.

4. Quando usar — e quando não usar

✓ Use SDD

- **Projeto novo do zero.** A spec orienta decisões desde o primeiro commit, sem dívida de contexto acumulada.
- **Feature complexa nova.** Múltiplos endpoints, regras de negócio, integrações externas.
- **Requisitos não-negociáveis.** Segurança, LGPD/GDPR, SLAs, compliance, limites de performance.
- **Time com mais de um dev.** Spec vira contrato entre front, back e QA.
- **Projeto mantido por meses.** Custo de escrever spec << custo de re-explicar contexto recorrente.
- **Integração com legado.** Documentar o que o código novo deve e **não deve** fazer protege contra quebras silenciosas.

- **Sistemas com forte uso de IA.** Quanto mais o agente gera código, mais a spec paga o custo de existir.

✗ Não use SDD

- **Scripts descartáveis, one-offs, crons únicos.** Vibe coding é mais rápido e adequado.
- **Prototipagem exploratória.** Spec congela decisões que você ainda precisa testar empiricamente.
- **Projetos muito curtos** (≤ 3 dias solo). Overhead não compensa.
- **Times sem disciplina para manter spec atualizada.** Spec desatualizada é pior que vibe coding — cria ilusão de controle.

5. EARS — os cinco padrões completos

EARS (Easy Approach to Requirements Syntax) usa **cinco padrões de sentença** que cobrem todos os tipos de requisito. Apresentar apenas o **WHEN/SHALL** é um erro comum de simplificação — força o autor a distorcer requisitos de estado, restrições contínuas e tratamento de erro em formato event-driven, perdendo precisão.

5.1 Ubiquitous (sempre ativo)

Regra incondicional, sempre verdadeira no sistema.

Sintaxe:

```
THE SYSTEM SHALL <ação>
```

Quando usar: invariantes, restrições globais, propriedades não-funcionais.

Exemplos:

```
THE SYSTEM SHALL armazenar todas as senhas usando bcrypt com cost factor mínimo de 12.
```

```
THE SYSTEM SHALL responder a 95% das requisições em menos de 300ms (P95).
```

```
THE SYSTEM SHALL registrar todas as alterações em registros de auditoria com user_id,  
timestamp e diff.
```

5.2 Event-driven (orientado a evento)

Ação disparada por um gatilho discreto.

Sintaxe:

```
WHEN <gatilho> THE SYSTEM SHALL <ação>
```

Quando usar: reações a entradas, chamadas de API, eventos de domínio.

Exemplos:

```
WHEN um usuário envia POST /login com credenciais válidas  
THE SYSTEM SHALL retornar token JWT com expiração de 60 minutos  
AND registrar o timestamp em users.last_login.
```

```
WHEN um pagamento é confirmado pelo gateway  
THE SYSTEM SHALL alterar o status do pedido para "paid"  
AND publicar evento order.paid no message bus.
```

5.3 State-driven (orientado a estado)

Ação válida **enquanto** um estado contínuo persiste.

Sintaxe:

```
WHILE <estado> THE SYSTEM SHALL <ação>
```

Quando usar: comportamentos que dependem de modo/contexto sustentado, não de um evento pontual.

Exemplos:

```
WHILE o usuário possuir sessão ativa  
THE SYSTEM SHALL exibir o menu autenticado em todas as páginas.  
  
WHILE o circuit breaker da integração com o gateway estiver aberto  
THE SYSTEM SHALL rejeitar novas tentativas de pagamento com HTTP 503  
AND retornar Retry-After header com 30 segundos.  
  
WHILE a aplicação estiver em modo manutenção  
THE SYSTEM SHALL responder apenas a GET /health.
```

5.4 Optional feature (recurso opcional)

Comportamento ativado por flag, plano ou configuração.

Sintaxe:

```
WHERE <feature/condição habilitada> THE SYSTEM SHALL <ação>
```

Quando usar: feature flags, planos premium, capacidades opcionais.

Exemplos:

```
WHERE o usuário tiver plano "Pro" ativo  
THE SYSTEM SHALL permitir até 50 projetos simultâneos  
AND exibir badge "Pro" no perfil.  
  
WHERE a feature flag "two_factor_auth" estiver habilitada para o tenant  
THE SYSTEM SHALL exigir verificação por TOTP no fluxo de login.  
  
WHERE o cliente enviar header X-API-Version: 2  
THE SYSTEM SHALL serializar respostas no schema v2.
```

5.5 Unwanted behavior (comportamento indesejado)

Tratamento explícito de erros, exceções e cenários adversos. **Esse é o padrão mais negligenciado** — e a fonte principal de bugs que escapam ao SDD mal aplicado.

Sintaxe:

```
IF <condição indesejada> THEN THE SYSTEM SHALL <ação>
```

Quando usar: erros, falhas, abuso, edge cases, fallbacks.

Exemplos:

```
IF o token JWT estiver expirado ou inválido
THEN THE SYSTEM SHALL retornar HTTP 401 com código de erro "token_expired"
AND NOT executar a operação requisitada.

IF mais de 5 tentativas de login falharem em 10 minutos para o mesmo e-mail
THEN THE SYSTEM SHALL bloquear o login do e-mail por 15 minutos
AND registrar evento security.bruteforce_suspected.

IF o webhook do gateway retornar status diferente de 2xx
THEN THE SYSTEM SHALL re-enfileirar a entrega com backoff exponencial (1s, 2s, 4s, 8s,
16s)
AND marcar como dead-letter após 5 falhas consecutivas.
```

5.6 Combinações (Complex Requirements)

EARS permite combinar gatilhos. Use com parcimônia — quando virar mais de duas condições, fatorize em requisitos separados.

```
WHEN POST /transfer é recebido
WHILE o usuário estiver autenticado
WHERE o saldo for suficiente
THE SYSTEM SHALL debitar o valor da conta de origem
AND creditar na conta de destino
AND registrar a transação no extrato.

IF o saldo for insuficiente
THEN THE SYSTEM SHALL retornar HTTP 422 com erro "insufficient_funds"
AND NOT alterar nenhum saldo.
```

5.7 Tabela de decisão rápida

Você precisa expressar...	Use
Uma regra sempre verdadeira	Ubiquitous
Uma reação a um evento/chamada	Event-driven (WHEN)
Um comportamento durante um estado	State-driven (WHILE)
Uma capacidade opcional/flagged	Optional feature (WHERE)
Um erro, falha ou edge case	Unwanted behavior (IF/THEN)

6. Anti-padrões em EARS

Erros comuns que neutralizam o ganho do SDD:

✗ Misturar "o quê" com "como"

```
WHEN usuário faz login
THE SYSTEM SHALL chamar bcrypt.compare() e gerar JWT com jsonwebtoken e salvar em Redis
```

Problema: spec amarrada a implementação. Substituir Redis por outra coisa "muda a spec", o que é absurdo.

Correto: spec descreve comportamento observável; implementação fica no plan.

```
WHEN usuário envia POST /login com credenciais válidas
THE SYSTEM SHALL retornar token de sessão com expiração de 60min
AND invalidar tokens anteriores do mesmo usuário.
```

✗ Forçar tudo em **WHEN**

```
WHEN o usuário está logado THE SYSTEM SHALL exibir avatar.
```

Problema: "estar logado" é estado, não evento. Use **WHILE**.

```
WHILE o usuário estiver autenticado THE SYSTEM SHALL exibir avatar no header.
```

✗ **SHALL** vago

```
THE SYSTEM SHALL ter boa performance.
THE SYSTEM SHALL ser seguro.
```


Problema: não-testável. Vira bullet de proposta comercial.

THE SYSTEM SHALL responder a 95% das requisições GET em menos de 200ms (P95) sob carga de até 100 RPS.

❌ Esquecer do **IF/THEN**

A spec sem padrão **Unwanted** cobre só o "happy path". Todo bug de produção mora no que ela não descreve. Regra: para cada **WHEN**, pergunte "e se falhar?" e escreva o **IF** correspondente.

❌ Confundir **AND** com nova cláusula

AND dentro de uma cláusula EARS é **payload da mesma ação**. Se você precisa de outra ação **condicional**, escreva nova cláusula.

```
# RUIM — duas regras coladas
WHEN compra é concluída THE SYSTEM SHALL enviar e-mail AND IF o usuário optou por SMS THE SYSTEM SHALL enviar SMS.

# BOM — duas regras separadas
WHEN compra é concluída THE SYSTEM SHALL enviar e-mail de confirmação.

WHEN compra é concluída
WHERE o usuário tiver opt-in para SMS habilitado
THE SYSTEM SHALL enviar SMS de confirmação.
```

7. Estrutura de pastas

Existem duas formas válidas — escolha conscientemente.

7.1 Versão simplificada (didática, customizada)

Útil para times pequenos ou projetos que não usam o Spec Kit oficial. A estrutura é criada uma vez na raiz e versionada com o código:

```
meu-projeto/
├── .cursor/
│   └── rules/
│       └── sdd.mdc          ← instruções globais para o agente
├── .specify/
│   ├── specs/              ← O QUÊ cada feature faz
│   │   └── auth-api.md
│   ├── plans/              ← EM QUE ORDEM implementar
│   │   └── auth-api.md
│   └── tasks/              ← cada UNIDADE atômica de trabalho
│       └── auth-api.md
└── src/
```

A separação entre `.cursor/` (configuração do editor) e `.specify/` (conteúdo do projeto) é intencional.

7.2 Versão oficial GitHub Spec Kit

Se você instalar a CLI oficial (`uvx --from git+https://github.com/github/spec-kit.git specify init`), a estrutura é diferente:

```
meu-projeto/
├── .specify/
│   ├── templates/          ← templates de spec, plan, tasks
│   ├── scripts/            ← shell/PowerShell scripts auxiliares
│   └── memory/
│       └── constitution.md ← princípios não-negociáveis do projeto
├── specs/
│   └── 001-auth-api/        ← uma pasta por feature, numerada
│       ├── spec.md
│       ├── plan.md
│       ├── tasks.md
│       └── contracts/       ← schemas, OpenAPI, etc.
├── .github/                ← prompts do Copilot (se Copilot)
│   └── prompts/
└── src/
```

Diferenças chave: - **Pasta por feature** (`specs/001-auth-api/`), não três pastas paralelas. - `constitution.md` define princípios duros do projeto que toda spec/plan deve respeitar. - **Numeração sequencial** (001, 002...) liga ao branch git.

Recomendação: comece com a versão simplificada para entender o fluxo. Migre para Spec Kit quando o time amadurecer e quiser slash commands (`/speckit.specify` , `/speckit.plan` , `/speckit.tasks` , `/speckit.implement`) e checks automatizados (`/speckit.analyze` , `/speckit.clarify`).

8. Configuração por ferramenta

A pasta `.specify/` é igual em todas. **Só o arquivo de regras globais muda.**

Ferramenta	Arquivo de regras	Particularidades
Cursor (≥2.2)	<code>.cursor/rules/sdd.mdc</code>	Requer frontmatter YAML
Cursor (legacy)	<code>.cursorrules</code> (raiz)	Deprecado, ainda funciona
Claude Code	<code>CLAUDE.md</code> (raiz)	Markdown puro; também suporta <code>AGENTS.md</code> (padrão aberto)
VSCode + Copilot	<code>.github/copilot-instructions.md</code>	Markdown; Spec Kit adiciona <code>.github/prompts/*.prompt.md</code>
VSCode + Cline	<code>.clinerules</code> (raiz)	Markdown puro
VSCode + Continue	<code>config.json</code> → <code>contextProviders</code>	JSON
Aider	<code>.aider.conf.yml</code> + <code>CONVENTIONS.md</code>	YAML config + markdown

8.1 Exemplo `.cursor/rules/sdd.mdc` (Cursor 2.2+)

⚠ **Importante:** a partir do Cursor 2.2, arquivos `.mdc` exigem **frontmatter YAML** para o Cursor decidir quando aplicá-los. Sem isso, a regra pode não disparar como você espera.

```

---
description: "SDD workflow rules – leitura obrigatória da spec antes de qualquer task"
globs:
alwaysApply: true
---

# SDD Rules

Antes de implementar qualquer feature:

1. Leia a spec correspondente em `.specify/specs/<feature>.md` (ou `specs/NNN-<feature>/spec.md` no Spec Kit).
2. Leia o plano em `.specify/plans/<feature>.md`, se existir.
3. Execute APENAS a task indicada em `.specify/tasks/<feature>.md`, na seção correspondente.
4. Valide o resultado contra a spec ao final – para cada `SHALL` da spec relacionado à task, confirme que a implementação atende.

## Regras duras

- NUNCA implemente além do escopo da task atual.
- Se a spec for ambígua, pergunte antes de assumir. Não invente requisito.
-
Se a implementação exigir alteração da spec, ALERTE antes – spec e código mudam no mesmo PR, nunca um sem o outro.
- Para cada `WHEN`/`WHILE`/`WHERE`/`IF` da spec, deve existir teste correspondente. Se não existir, gere o teste antes do código.

## Convenções de código (preencha conforme o projeto)

- Linguagem principal: <ex: TypeScript estrito>
- Padrão de erro: <ex: Result<T, E> em vez de exceções>
- Cobertura mínima: <ex: 80%>

```

Campo do frontmatter	Função
<code>description</code>	Texto que o agente vê quando decide se aplica a regra (modo "Agent Requested")
<code>globs</code>	Padrões de arquivo que disparam a regra automaticamente (modo "Auto Attached")
<code>alwaysApply: true</code>	Aplica a regra em toda sessão, independente de arquivo ou contexto

Para regras de processo (como SDD), `alwaysApply: true` é o correto. Para regras específicas de stack (ex: "padrões React"), use `globs: ["**/*.tsx"]`.

8.2 Exemplo `CLAUDE.md` (Claude Code)

```
# Projeto X – Convenções e SDD

## Workflow obrigatório

Toda implementação segue Spec-Driven Development:

1. Antes de codar, leia `.specify/specs/<feature>.md`.
2. Implemente apenas a task atual indicada em `.specify/tasks/<feature>.md`.
3. Para cada `SHALL` da spec coberto pela task, escreva ou atualize teste correspondente.
4. Valide contra a spec – não amplie escopo.

## Stack

- Backend: <...>
- Frontend: <...>
- Banco: <...>

## Regras não-negociáveis

- Spec e código mudam no mesmo PR.
- Mudança de comportamento observável SEM atualização de spec = revert obrigatório.
```

9. Workflow completo

Passo 1 — Constituição do projeto (opcional, mas recomendado)

Antes da primeira spec, defina os **princípios não-negociáveis** em `.specify/memory/constitution.md` (ou equivalente):

```
# Constituição do Projeto

## Princípios duros

1. Toda resposta de API segue contrato OpenAPI versionado.
2. Nenhum endpoint público pode existir sem rate limiting.
3. Dados pessoais são criptografados em repouso (AES-256) e em trânsito (TLS 1.3+).
4. Nenhuma migration destrutiva sem janela de rollback de 24h.
5. Cobertura de testes mínima: 80% global, 95% em módulos de billing e auth.

## Política de IA

- Agente de IA não pode gerar código que altere schema sem aprovação humana explícita.
- Agente não pode introduzir nova dependência runtime sem justificativa em PR.
```

A constituição é lida **antes** da spec, em toda sessão. Funciona como o "linter conceitual" do projeto.

Passo 2 — Escrever a spec

Crie um arquivo Markdown por feature. Nome em kebab-case, descritivo.

Localização: - Simplificado: `.specify/specs/<feature>.md` - **Spec** **Kit:**
`specs/NNN-<feature>/spec.md`

Estrutura recomendada:

```
# Spec: <Nome da Feature>

## Contexto
<1-2 parágrafos: o problema que essa feature resolve, qual usuário, qual sistema.>

## Glossário
- **<termo>**: <definição>

## Requisitos (formato EARS)

### Ubiquitous
THE SYSTEM SHALL ...

### Event-driven
WHEN ... THE SYSTEM SHALL ...

### State-driven
WHILE ... THE SYSTEM SHALL ...

### Optional feature
WHERE ... THE SYSTEM SHALL ...

### Unwanted behavior
IF ... THEN THE SYSTEM SHALL ...

## O que NÃO faz (não-requisitos explícitos)
- Não envia e-mails (responsabilidade do serviço X)
- Não persiste histórico (apenas estado atual)

## Restrições não-funcionais
- Performance: ...
- Segurança: ...
- Compliance: ...

## Critérios de aceite globais
- [ ] Todos os WHEN/SHALL têm testes correspondentes passando
- [ ] Cobertura ≥ 80% no módulo
- [ ] Documentação OpenAPI atualizada
```

Passo 3 — Gerar o plano

Com a spec aberta, peça ao agente:

Com base em `@.specify/specs/auth-api.md`, gere um plano de implementação ordenado em `.specify/plans/auth-api.md`.

O plano deve:

- Listar etapas em ordem de dependência técnica.
- Para cada etapa, indicar quais cláusulas EARS da spec ela cobre.
- Não incluir tarefas que não estejam justificadas por algum SHALL da spec.
- Identificar pré-requisitos de infraestrutura (migrations, novos serviços, etc).

Não escreva código nesse passo.

Revise o plano gerado. Corrija ordem, remova etapas que extrapolaram o escopo, garanta rastreabilidade plan → spec.

Passo 4 — Decompor em tasks atômicas

Com base em `@.specify/plans/auth-api.md` e `@.specify/specs/auth-api.md`, decomponha em tasks atômicas em `.specify/tasks/auth-api.md`.

Cada task deve ter:

- ID (1.1, 1.2, ...) e título
- Descrição (uma frase)
- Cláusulas EARS cobertas (referência direta à spec)
- Dependências explícitas (outras task IDs)
- Critério de aceite binário (passou/não passou)
- Estimativa em tamanho relativo (S/M/L), onde:
 - S = cabe em uma sessão de até 30min
 - M = cabe em uma sessão de até 90min
 - L = quebrar em sub-tasks

Não crie task L. Se algo for L, quebre.

Passo 5 — Implementar task por task

Forneça ao agente **apenas** a task atual e a spec correspondente. Nunca o projeto inteiro, nunca várias tasks de uma vez.

Implemente `@.specify/tasks/auth-api.md Task 1.1`.

Siga as restrições de `@.specify/specs/auth-api.md` e os princípios de `@.specify/memory/constitution.md`.

Antes de implementar:

1. Liste os SHALLs cobertos por essa task.
2. Para cada SHALL, escreva o teste correspondente (que deve falhar inicialmente).
3. Implemente o mínimo para os testes passarem.
4. Ao final, confirme: critério de aceite atendido? Sim/Não, com evidência.

⚠ **Atenção:** isolamento é o que mantém erros pequenos. Combinar tasks "para economizar tempo" é o caminho mais rápido para perder controle.

Passo 6 — Verificar contra a spec

Duas camadas de verificação:

6.1 Testes derivados da spec

Regra: cada cláusula EARS gera no mínimo um caso de teste.

Padrão EARS	Tipo de teste
Ubiquitous	Teste de invariante (rodado em toda suíte) ou property test
Event-driven	Teste de integração do happy path
State-driven	Teste com setup/teardown de estado
Optional feature	Teste parametrizado por flag/plano
Unwanted behavior	Teste de erro/edge case (obrigatório)

6.2 Revisão humana focada

A pergunta é "esse código faz exatamente o que a spec diz — nem mais, nem menos?". Você pode pedir uma revisão preliminar ao próprio agente:

```
Compare @src/auth/ com @.specify/specs/auth-api.md.
```

Liste:

1. Cláusulas SHALL da spec NÃO implementadas no código.
2. Comportamentos no código NÃO previstos na spec (sobre-implementação).
3. Divergências entre o que a spec exige e o que o código faz.

Para cada item, cite arquivo:linha do código e seção da spec.

Passo 7 — Manter a spec atualizada

🚨 **Regra inegociável:** spec e código mudam no mesmo Pull Request. Sempre.

Implemente no CI:


```
# .github/workflows/spec-sync.yml (exemplo conceitual)
- name: Verify spec sync
  run: |
    if git diff --name-only origin/main | grep -E '^src/' && \
      ! git diff --name-only origin/main | grep -E '^\.specify/specs/'; then
      echo "::error::Código alterado sem atualização de spec. PR bloqueado."
      exit 1
    fi
```

Spec desatualizada é pior que ausência de spec — cria ilusão de controle enquanto o time toma decisões baseadas em informação errada.

10. Exemplo end-to-end

10.1 `.specify/specs/auth-api.md`

```
# Spec: API de Autenticação de Usuários

## Contexto
Endpoints REST para autenticação via e-mail/senha, com tokens JWT de curta duração
e refresh tokens rotacionáveis. Suporte a logout e proteção contra brute force.

## Glossário
- **Access token**: JWT de curta duração (60min), usado em Authorization header.
- **Refresh token**: opaque token (UUID v4) armazenado em DB, usado para obter novo access token.

## Requisitos

### Ubiquitous
THE SYSTEM SHALL armazenar senhas usando Argon2id com parâmetros mínimos
m=65536, t=3, p=4.

THE SYSTEM SHALL retornar 95% das respostas de POST /login em menos de 300ms (P95)
sob carga de até 50 RPS.

THE SYSTEM SHALL exigir TLS 1.3+ em todos os endpoints públicos.

### Event-driven
WHEN um cliente envia POST /login com e-mail e senha válidos
THE SYSTEM SHALL retornar HTTP 200 com payload { access_token, refresh_token, expires_in }
AND atualizar users.last_login_at com timestamp atual
AND registrar evento auth.login_success no log estruturado.

WHEN um cliente envia POST /refresh com refresh_token válido e não expirado
THE SYSTEM SHALL retornar novo access_token e novo refresh_token
AND invalidar o refresh_token anterior (rotação).

WHEN um cliente envia POST /logout com Authorization válido
THE SYSTEM SHALL invalidar o access_token atual
AND invalidar todos os refresh_tokens do usuário
AND retornar HTTP 204.

WHEN um cliente envia GET /users/{id} com Authorization válido
THE SYSTEM SHALL retornar HTTP 200 com { id, email, created_at, last_login_at }
do usuário com id correspondente.

### State-driven
WHILE um access_token estiver válido e não expirado
THE SYSTEM SHALL aceitar requisições autenticadas nos endpoints protegidos
AND incluir user_id no contexto da request.

WHILE a aplicação estiver em modo manutenção (env MAINTENANCE_MODE=true)
THE SYSTEM SHALL responder HTTP 503 em todos endpoints exceto GET /health.

### Optional feature
WHERE o tenant tiver feature flag "two_factor_auth" habilitada
THE SYSTEM SHALL exigir código TOTP no POST /login após validação de senha
AND retornar HTTP 200 com challenge_token de validade 5min em vez de access_token.

### Unwanted behavior
IF credenciais inválidas forem enviadas em POST /login
THEN THE SYSTEM SHALL retornar HTTP 401 com { error: "invalid_credentials" }
AND NÃO revelar se o e-mail existe no sistema (mesma resposta para e-mail inexistente).
```

```
IF mais de 5 tentativas de login falharem para o mesmo e-mail em 10 minutos
THEN THE SYSTEM SHALL bloquear novas tentativas desse e-mail por 15 minutos
AND retornar HTTP 429 com Retry-After: 900
AND registrar evento security.bruteforce_suspected.
```

```
IF o access_token estiver expirado, malformado ou ausente em endpoint protegido
THEN THE SYSTEM SHALL retornar HTTP 401 com { error: "token_invalid" }
AND NÃO executar a operação.
```

```
IF GET /users/{id} for chamado com id de usuário inexistente
THEN THE SYSTEM SHALL retornar HTTP 404 com { error: "user_not_found" }.
```

```
IF um refresh_token for usado mais de uma vez (replay)
THEN THE SYSTEM SHALL invalidar TODOS os refresh_tokens do usuário
AND retornar HTTP 401 com { error: "token_reuse_detected" }
AND registrar evento security.token_replay.
```

O que NÃO faz

- Não envia e-mails de recuperação de senha (responsabilidade do serviço `notifications`).
- Não persiste histórico de sessões anteriores (apenas tokens ativos).

-

Não gerencia cadastro de novos usuários (responsabilidade do endpoint POST /signup, fora desta spec).

Restrições não-funcionais

- Argon2id com parâmetros mínimos definidos acima.
- Refresh tokens rotacionados a cada uso.
- Detecção de replay em refresh tokens é obrigatória.
- Logs estruturados em JSON, campos obrigatórios: timestamp, event, user_id, ip, user_agent.

Critérios de aceite globais

- [] Todos os WHEN/SHALL têm testes de integração passando.
- [] Todos os IF/THEN têm testes de erro correspondentes.
- [] Suite de testes roda em < 30s.
- [] Cobertura no módulo auth ≥ 95%.
- [] OpenAPI atualizado em `docs/openapi.yaml`.
- [] Carga simulada de 50 RPS confirma P95 < 300ms.

10.2 `.specify/plans/auth-api.md` (resumo)

Plano: API de Autenticação

Ordem de implementação

1. **Infra** – schema users + refresh_tokens + audit_log; migration; índices.
2. **Hash & verificação** – wrapper Argon2id, testes unitários (cobre Ubiquitous 1).
3. **POST /login happy path** – sem 2FA, sem rate limit (cobre Event-driven 1).
4. **Middleware de autenticação** – validação de access_token (cobre State-driven 1, IF token_invalid).
5. **GET /users/{id}** – depende do middleware (cobre Event-driven 4, IF user_not_found).
6. **POST /refresh com rotação** – inclui detecção de replay (cobre Event-driven 2, IF token_reuse).
7. **POST /logout** – invalidação total (cobre Event-driven 3).
8. **Rate limiting de login** – bucket por e-mail (cobre IF bruteforce).
9. **2FA opcional via feature flag** – challenge_token + TOTP (cobre Optional feature 1).
10. **Modo manutenção** – middleware global (cobre State-driven 2).
11. **Observabilidade** – logs estruturados, métricas P95 (cobre Ubiquitous 2).

Pré-requisitos de infra

- Redis para rate limit e blacklist de access tokens.
- Chave de assinatura JWT em secrets manager (não em env var de container).

10.3 `.specify/tasks/auth-api.md` (trecho)

```
## Task 1.1 – Schema e migration de users
Cobre: (nenhum SHALL diretamente; pré-requisito)
Dependência: nenhuma
Tamanho: S
Aceite:
- [ ] Migration cria tabela users (id UUID, email UNIQUE NOT NULL, password_hash, last_login_at, created_at).
- [ ] Migration aplicada em ambiente de teste com sucesso.
- [ ] Rollback testado.

## Task 1.2 – Wrapper Argon2id
Cobre: Ubiquitous 1 ("armazenar senhas usando Argon2id...")
Dependência: nenhuma
Tamanho: S
Aceite:
- [ ] Função hash(password) → string usa m=65536, t=3, p=4.
- [ ] Função verify(password, hash) → boolean.
- [ ] Teste: hash de "abc" verifica positivo; hash de "abc" não verifica "xyz".
- [ ] Teste: dois hashes consecutivos da mesma senha são diferentes (salt aleatório).

## Task 1.3 – POST /login happy path
Cobre: Event-driven 1 (login com credenciais válidas)
Dependência: 1.1, 1.2
Tamanho: M
Aceite:
- [ ] Endpoint retorna 200 com { access_token, refresh_token, expires_in: 3600 }.
- [ ] access_token é JWT válido assinado, expira em 60min.
- [ ] refresh_token é UUID v4 persistido em refresh_tokens.
- [ ] users.last_login_at atualizado.
- [ ] Evento auth.login_success no log estruturado.
- [ ] Teste de integração cobrindo o fluxo completo.

## Task 1.4 – IF credenciais inválidas em POST /login
Cobre: Unwanted 1 (invalid_credentials)
Dependência: 1.3
Tamanho: S
Aceite:
- [ ] Senha errada para e-mail existente → HTTP 401 { error: "invalid_credentials" }.
- [ ] E-mail inexistente → HTTP 401 { error: "invalid_credentials" } (mesma resposta).
- [ ] Tempo de resposta dos dois casos não difere em mais de 50ms (proteção contra timing attack).
- [ ] Não há nenhum log que diferencie os dois casos no nível INFO ou superior.

## Task 1.5 – Middleware de autenticação
Cobre: State-driven 1, Unwanted 3 (token_invalid)
Dependência: 1.3
Tamanho: M
Aceite:
- [ ] Authorization válido → request prossegue, user_id no contexto.
- [ ] Authorization ausente → 401 { error: "token_invalid" }.
- [ ] Authorization malformado → 401 { error: "token_invalid" }.
- [ ] Authorization expirado → 401 { error: "token_invalid" }.
- [ ] Endpoint POST /login NÃO passa pelo middleware (whitelist).

# (... continua até Task 1.11)
```

11. Decomposição em tasks atômicas

Uma task bem feita tem **quatro** propriedades obrigatórias (não três):

1. **Cabe em uma única sessão sem recarregar contexto** — operacionalmente: ≤ 90 minutos de trabalho ativo, $\leq \sim 400$ LOC modificadas, $\leq \sim 8k$ tokens de contexto necessário.
2. **Verificável de forma binária** — critério de aceite passou ou não passou, sem zona cinzenta.
3. **Independente, ou com dependência explicitamente declarada** — outras tasks IDs listadas.
4. **Rastreável até a spec** — cita quais cláusulas EARS cobre. Task que não cobre nenhum **SHALL** é candidata a remoção.

Sinais de que a task está mal decomposta

Sinal	Causa provável	O que fazer
Critério de aceite tem "e", "também", "além disso"	Task agrega múltiplas responsabilidades	Quebrar em duas
Task não cita cláusula da spec	Sobre-implementação ou tarefa de infra disfarçada	Mover para "pré-requisitos" ou eliminar
Você precisa do projeto inteiro aberto para implementar	Falta isolamento	Quebrar até caber com 2-3 arquivos abertos
Dependência cíclica entre tasks	Plan mal ordenado	Refazer o plan
Aceite mensurado em "horas estimadas"	Task ainda em escopo, não em entrega	Reescrever aceite como observável binário

12. Verificação contra a spec

12.1 Cobertura de cláusulas

Para cada cláusula EARS da spec, garanta: - Existe ao menos uma task que a cobre. - Existe ao menos um teste que valida a cláusula. - O teste falha se o comportamento for removido (mutation testing opcional, mas valioso).

Você pode automatizar isso com um script simples:

```
# Pseudo-script de verificação
for cláusula in parse_ears(spec.md):
    if not search_tests(cláusula.id):
        warn(f"Cláusula {cláusula.id} sem teste correspondente")
```

12.2 Detecção de drift

Pergunta a fazer ao agente em revisão:

Compare @src/ com @.specify/specs/auth-api.md.

Em formato de tabela, liste:

| Cláusula EARS | Implementada? | Teste correspondente? | Arquivo/linha |

Onde a coluna "Implementada?" pode ser: SIM / NÃO / PARCIAL.

Onde "PARCIAL", explique o que falta.

12.3 Sobre-implementação

A spec define **também o que não fazer**. Verifique se o código não adicionou: - Endpoints não previstos. - Campos extras em respostas. - Comportamentos "úteis" mas fora do escopo.

Sobre-implementação parece inofensiva, mas vira contrato implícito que você terá de manter para sempre.

13. Governança

Artefato	O que define	Pergunta que responde	Quem mantém
<code>constitution.md</code>	Princípios duros do projeto	Quais regras nenhuma feature pode violar?	Tech lead + arquiteto
<code>.specify/specs/* .md</code>	Comportamento observável	O quê?	Dev + Product Owner
<code>.specify/plans/* .md</code>	Ordem técnica de execução	Em que ordem?	Dev líder da feature
<code>.specify/tasks/* .md</code>	Unidades atômicas de trabalho	Como dividir?	Dev responsável
<code>src/**</code>	Implementação	Como executa?	Dev + agente de IA
<code>tests/**</code>	Validação da spec	Foi implementado certo?	Dev + QA

Política de PR

Toda PR que altera comportamento observável deve conter:

- [] Atualização da spec (`.specify/specs/`) se houve mudança de requisito.
- [] Atualização do plan e tasks se houve mudança de escopo.
- [] Testes correspondentes a cada novo/alterado `SHALL`.
- [] Checklist de cláusulas cobertas no corpo do PR.

PRs que só alteram código sem tocar spec passam por revisão extra: ou é refactor puro (não mudou comportamento), ou está burlando o processo.

14. Métricas e sinais de saúde do processo

Como saber se SDD está funcionando no seu time, e não virou teatro:

Sinais positivos

- **Tempo médio de revisão de PR caindo** ao longo do tempo (revisor sabe contra o quê comparar).
- **Bugs de regressão caindo** (testes derivados da spec pegam regressões).
- **Onboarding de novo dev acelerando** (spec serve como mapa do território).
- **Sessões de IA produzindo código aceito sem retrabalho** acima de ~80%.
- **Cobertura de teste estável e crescente** sem esforço dedicado.

Sinais negativos (red flags)

- **Spec sendo atualizada APÓS o código** com frequência → política de PR não está sendo enforced.
- **Tasks "L" aparecendo na lista** → decomposição falhando.
- **PRs com "TODO: atualizar spec depois"** → começo do fim.
- **Spec ficando longa demais** (>500 linhas para uma feature) → quebrar em sub-specs.
- **Dev ignorando o agente e codando direto** → ou o agente está ruim, ou a spec está vaga demais para o agente usar.
- **"Constitution.md" virando wish-list** → manter no máximo 10 princípios, todos verificáveis.

Métrica composta sugerida

SDD Health Score (qualitativo, revisar mensalmente):

Dimensão	Pergunta	Peso
Sincronização	% de PRs com spec atualizada no mesmo commit do código	30%
Rastreabilidade	% de tasks que citam cláusula EARS	20%
Cobertura	% de cláusulas com teste correspondente	25%
Aderência	% de PRs onde o agente seguiu o escopo da task	15%
Reuso	% de specs reutilizáveis (não amarradas à implementação)	10%

Se cair abaixo de 70%, revisar processo em retrospectiva.

15. Referência rápida

Artefatos

Artefato	Finalidade
<code>.cursor/rules/sdd.mdc</code> (Cursor)	Instrui agente; frontmatter YAML obrigatório
<code>CLAUDE.md</code> (Claude Code)	Instrui agente; markdown puro
<code>.github/copilot-instructions.md</code> (Copilot)	Instrui agente
<code>.specify/memory/constitution.md</code>	Princípios não-negociáveis do projeto
<code>.specify/specs/*.md</code> ou <code>specs/NNN-<feat>/spec.md</code>	O QUÊ (formato EARS)
<code>.specify/plans/*.md</code> ou <code>specs/NNN-<feat>/plan.md</code>	EM QUE ORDEM
<code>.specify/tasks/*.md</code> ou <code>specs/NNN-<feat>/tasks.md</code>	UNIDADE atômica com critério de aceite

Os cinco padrões EARS

Padrão	Sintaxe	Para...
Ubiquitous	<code>THE SYSTEM SHALL <ação></code>	Invariantes globais
Event-driven	<code>WHEN <gatilho> THE SYSTEM SHALL <ação></code>	Reações a eventos
State-driven	<code>WHILE <estado> THE SYSTEM SHALL <ação></code>	Comportamento durante estado
Optional feature	<code>WHERE <feature/condição> THE SYSTEM SHALL <ação></code>	Capacidades opcionais
Unwanted behavior	<code>IF <condição> THEN THE SYSTEM SHALL <ação></code>	Erros, edge cases

Camadas de responsabilidade

Camada	Pergunta	Quem
Constitution	Quais princípios são invioláveis?	Tech lead
Spec	O quê?	Dev + PO
Plan	Em que ordem?	Dev líder
Tasks	Como dividir?	Dev da feature
Código	Como implementar?	Dev + IA
Testes	Foi implementado certo?	Dev + QA

Workflow em uma linha

Constitution → Spec (EARS) → Plan → Tasks atômicas → Implementar com IA, uma task por vez → Verificar contra a spec → PR com spec e código juntos.

Frase para fixar

Vibe coding otimiza o prompt. SDD otimiza a fundação. EARS otimiza a sentença. SDD realocaliza o problema dos requisitos; EARS o ataca onde ele mora: na frase. Disciplina dos três stackados é o multiplicador. Nenhum deles isolado é.

Departamento WEB · ETAPA Educacional · Edição 2026 · Baseado em GitHub Spec Kit (set/2025), AWS Kiro (jul/2025), padrão EARS (Mavin, Rolls-Royce, 2009) e práticas consolidadas em 2025–2026.